

SOFTWARE REQUIREMENTS SPECIFICATION

World In Brief

AI News Intelligence Platform

IEEE 830-1998 Compliant Document

Version 1.0

July 16, 2026

Prepared for Academic Submission — Software Engineering Course Project

Revision History

Version	Date	Author	Description
1.0	July 16, 2026	Project Team — World In Brief	Initial release of the Software Requirements Specification.

Table of Contents

Revision History.....	1
Table of Contents	2
1. Introduction.....	5
1.1 Purpose.....	5
1.2 Document Conventions.....	5
1.3 Intended Audience and Reading Suggestions	5
1.4 Project Scope.....	6
1.5 References	6
2. Overall Description.....	7
2.1 Product Perspective	7
High-Level Architecture Flow.....	7
2.2 Product Features	7
2.3 User Classes and Characteristics.....	8
2.4 Operating Environment	8
2.5 Design and Implementation Constraints.....	8
2.6 Assumptions and Dependencies.....	8
3. System Features	10
3.1 Authentication Module.....	10
3.2 Personalized News Module	10
3.3 AI Intelligence Module.....	11
3.4 Trending Module.....	11
3.5 Notification Module	12
3.6 Bookmark Module.....	12
3.7 Feedback Module.....	13
3.8 Admin Module.....	13
4. Data Requirements and Database Design	15
Users	15
Articles.....	15
Categories.....	15
Bookmarks.....	15
ReadingHistory	16
Preferences.....	16
Notifications	16
AIConversation	16

AIConversationMessage	16
Feedback	17
TrendingTopics	17
Analytics	17
4.1 Entity Relationships	18
4.2 Normalization	18
5. External Interface Requirements	19
5.1 User Interfaces	19
5.2 Hardware Interfaces	19
5.3 Software Interfaces	19
5.4 Communication Interfaces	19
6. Non-Functional Requirements	20
6.1 Performance	20
6.2 Security	20
6.3 Reliability	20
6.4 Availability	20
6.5 Scalability	21
6.6 Maintainability	21
6.7 Portability	21
6.8 Accessibility	21
6.9 Privacy	21
6.10 Backup & Recovery	21
6.11 Logging	22
6.12 Monitoring	22
7. UML Modeling Support	23
7.1 Actors	23
7.2 Use Case Diagram	23
7.3 Class Diagram	23
7.4 Entity-Relationship (ER) Diagram	23
7.5 Sequence Diagrams	23
7.6 Activity Diagram	24
7.7 State Diagram	24
7.8 Component Diagram	24
7.9 Deployment Diagram	24
7.10 Package Diagram	24
8. Use Cases	25

9. User Stories	33
10. Operating Environment	35
10.1 Client Environment	35
10.2 Server Environment	35
10.3 Browser Support	35
10.4 Operating Systems	35
10.5 Network Requirements.....	35
10.6 Deployment Environment.....	35
11. Software Architecture.....	36
12. Future Scope.....	37

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for World In Brief, an AI-powered News Intelligence Platform. The document defines the scope, features, data model, interfaces, and quality attributes of the system in sufficient detail to guide design, implementation, testing, and evaluation of the platform as a software engineering project. It is intended to serve as the single source of truth for what the system shall do, and is written in conformance with the IEEE 830-1998 recommended practice for software requirements specifications.

1.2 Document Conventions

This document uses the keyword “shall” to denote a mandatory requirement and “should” to denote a recommended, non-mandatory capability. Functional requirements are numbered FR-1 through FR-55 and grouped by module. Non-functional requirements are numbered NFR-1 through NFR-34 and grouped by quality attribute. Use cases are numbered UC-01 through UC-24. The following abbreviations are used throughout the document:

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
RSS	Really Simple Syndication — the news feed format consumed from BBC
AI	Artificial Intelligence
LLM	Large Language Model
JWT	JSON Web Token
FR / NFR	Functional Requirement / Non-Functional Requirement
UC	Use Case
PK / FK	Primary Key / Foreign Key
3NF	Third Normal Form
WCAG	Web Content Accessibility Guidelines

1.3 Intended Audience and Reading Suggestions

This document is intended for the project development team (frontend, backend, and AI engineers), the course instructor and evaluators, and any future maintainers of the system. Readers interested in high-level scope should read Sections 1 and 2. Developers implementing specific modules should read Sections 3, 4, and 6. Database designers should read Section 5. Readers producing UML diagrams for the project should read Section 7. Testers and QA reviewers should focus on Sections 8 and 9, which contain the use cases and user stories used to derive test cases.

1.4 Project Scope

World In Brief aggregates live news from BBC RSS feeds and layers an AI Agent on top of the raw news feed to deliver a personalized, conversational, and intelligent reading experience. Unlike a conventional news aggregator, the system understands natural-language requests, summarizes and translates articles, classifies topics and sentiment, recommends content, and proactively delivers a personalized daily briefing. The platform is composed of a responsive web frontend, a Node.js/Express backend, and an AI Agent layer built around a local Llama 3.2 model served through Ollama. The scope of this document covers the authentication, personalized news, AI intelligence, trending, notification, bookmark, feedback, and administration modules, along with their supporting data model and system architecture. Native mobile applications and persistent long-term AI memory are explicitly out of scope for this release and are captured in Section 12, Future Scope.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- BBC News RSS feed documentation (public feed endpoints by category).
- Ollama documentation for local large language model serving.
- Meta Llama 3.2 model card and usage guidelines.
- Node.js and Express.js official documentation.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.

2. Overall Description

2.1 Product Perspective

World In Brief is a new, self-contained web application; it is not a component of an existing system, though it depends on the external BBC RSS feed as its source of raw news content and on a locally hosted Ollama instance for AI inference. The system follows a layered client-server architecture in which a browser-based frontend communicates with an Express.js backend over a REST API. The backend, in turn, delegates any AI-related request to an internal AI Agent, which itself is composed of a Planner, an Executor, a Memory component, and a News Tool. The following diagram summarizes the high-level data and control flow through the system.

High-Level Architecture Flow

Step	Component	Responsibility
1	User	Interacts with the web application through a browser.
2	Frontend	Renders the UI and sends requests to the backend API.
3	Express API	Authenticates requests and routes them to the appropriate service.
4	AI Agent	Receives natural-language or AI-feature requests from the API layer.
5	Planner	Decomposes the request into an ordered sequence of actions.
6	Executor	Carries out each planned action, invoking tools as required.
7	News Tool	Retrieves and filters articles from the BBC RSS feed.
8	Ollama (Llama 3.2)	Performs the underlying language-model inference.
9	Response	Formatted result is returned to the user through the API and frontend.

2.2 Product Features

At a high level, World In Brief provides the following categories of functionality, each elaborated as a system feature in Section 3:

- Account creation, authentication, and profile management.
- Category-based browsing, search, and a personalized news feed.
- An AI Agent offering conversational chat, daily briefs, summarization, translation, classification, sentiment analysis, headline generation, plain-language explanations, and multi-article comparison.
- Trending topics, popular articles, and personal reading analytics.
- Configurable breaking-news and digest notifications.
- Article bookmarking with offline reading support.
- User feedback, AI response rating, and bug reporting.
- An administrative console for user, content-source, and analytics management.

2.3 User Classes and Characteristics

The system recognizes three classes of users, distinguished by the privileges available to them:

User Class	Characteristics	Representative Capabilities
Guest User	Unauthenticated visitor; requires no technical expertise.	Browse categories, search news, view trending topics.
Registered User	Authenticated end user with a personal account; the primary user class.	All guest capabilities, plus personalization, AI features, bookmarks, notifications, and feedback.
Administrator	Trusted internal user responsible for platform operation.	All registered-user capabilities, plus user management, news-source management, analytics, and moderation.

2.4 Operating Environment

World In Brief operates as a web application accessed through a standard browser and is hosted on a Node.js-compatible server environment. Full operating-environment detail, including client and server specifications, browser support, and network requirements, is provided in Section 10, Operating Environment.

2.5 Design and Implementation Constraints

- The AI Agent must use a locally hosted Ollama instance running the Llama 3.2 model; no external, third-party LLM API may be used, to keep the system self-hosted and cost-free for the course project.
- The backend must be implemented using Node.js and the Express.js framework.
- The frontend must be implemented using HTML, CSS, and JavaScript, and must support both light and dark themes.
- The relational database schema must be normalized to at least Third Normal Form (3NF).
- All AI Agent invocations must pass through the Planner/Executor pipeline so that tool use remains auditable and testable.
- The system must remain functional for non-AI features (browsing, search, bookmarking) even if the AI Agent or Ollama service is unavailable.

2.6 Assumptions and Dependencies

- The BBC RSS feed remains publicly accessible and its schema does not change in a breaking way during the project timeline.
- A machine capable of running the Llama 3.2 model through Ollama is available for AI inference during development and demonstration.
- Users have a modern web browser and a stable internet connection.

- Course evaluation environments allow local installation of Node.js, a relational database, and Ollama.

3. System Features

3.1 Authentication Module

Description and Priority: Provides secure account creation, login, logout, and credential-recovery capabilities that underpin every personalized capability in the platform. Overall module priority: High.

Functional Requirements:

ID	Requirement	Priority
FR-1	The system shall allow a new user to register using a unique email address, username, and password.	High
FR-2	The system shall verify that the email address supplied at registration is not already associated with an existing account.	High
FR-3	The system shall allow a registered user to log in using a valid email/username and password combination.	High
FR-4	The system shall issue a signed session token (JWT) to the client upon successful authentication.	High
FR-5	The system shall allow a logged-in user to log out, invalidating the active session token.	High
FR-6	The system shall allow a user to request a password-reset link be sent to their registered email address.	High
FR-7	The system shall allow a user to set a new password using a valid, unexpired password-reset link.	High
FR-8	The system shall allow a user to update profile details, including display name, avatar, and preferred language.	Medium

3.2 Personalized News Module

Description and Priority: Builds a tailored reading experience by combining explicit user preferences, implicit reading behavior, and AI-generated recommendations. Overall module priority: High.

Functional Requirements:

ID	Requirement	Priority
FR-9	The system shall allow a registered user to select one or more favorite news categories.	High
FR-10	The system shall allow a registered user to remove a category from their list of favorites.	Medium
FR-11	The system shall record each article opened by a registered user in that user's reading history.	High
FR-12	The system shall allow a registered user to view a chronological list of their reading history.	Medium
FR-13	The system shall generate a personalized news feed ordered using the user's favorite categories and reading history.	High

ID	Requirement	Priority
FR-14	The system shall use the AI Agent to recommend articles related to a user's recent reading pattern.	High
FR-15	The system shall refresh a user's personalized feed automatically when new matching articles are ingested.	Medium
FR-16	The system shall allow a registered user to clear their entire reading history.	Low

3.3 AI Intelligence Module

Description and Priority: Wraps the Ollama-hosted Llama 3.2 model in an AI Agent (Planner, Executor, Memory, and News Tool) to deliver conversational assistance, summarization, translation, classification, and comparison features. Overall module priority: High.

Functional Requirements:

ID	Requirement	Priority
FR-17	The system shall provide an AI Chat Assistant capable of answering natural-language questions about current news.	High
FR-18	The AI Agent shall use a Planner component to decompose an incoming user request into an ordered execution plan.	High
FR-19	The AI Agent shall use an Executor component to carry out each step of the plan, including invoking the News Tool.	High
FR-20	The system shall generate a personalized AI Daily Brief summarizing the top stories relevant to a user, once per day.	High
FR-21	The system shall generate an AI-produced summary of any selected article on user request.	High
FR-22	The system shall translate a selected article into a user-selected target language using the AI model.	High
FR-23	The system shall automatically classify each ingested article into one or more topic categories using AI classification.	High
FR-24	The system shall perform sentiment analysis on ingested articles and expose the result (positive, neutral, negative) to the user.	Medium
FR-25	The system shall generate alternate AI-written headlines for a selected article on request.	Low
FR-26	The system shall provide a simplified, plain-language AI explanation of complex news articles on request.	Medium
FR-27	The system shall allow a user to select two or more articles and receive an AI-generated comparison of their content.	Medium
FR-28	The AI Agent shall retain short-term conversational memory for the duration of an active user session.	High

3.4 Trending Module

Description and Priority: Surfaces what is currently popular across the platform, both globally and for an individual user, using aggregated read statistics. Overall module priority: Medium.

Functional Requirements:

ID	Requirement	Priority
FR-29	The system shall compute trending topics based on aggregate article read counts within a rolling time window.	High
FR-30	The system shall display the most popular articles of the current day on the homepage.	Medium
FR-31	The system shall display the categories with the highest read volume over a configurable period.	Medium
FR-32	The system shall provide a registered user with reading analytics showing their category activity over time.	Low
FR-33	The system shall recompute trending topics at a configurable scheduled interval.	Medium

3.5 Notification Module

Description and Priority: Delivers timely, configurable alerts so users stay informed of breaking stories and daily summaries without needing to check the app manually. Overall module priority: Medium.

Functional Requirements:

ID	Requirement	Priority
FR-34	The system shall send a breaking-news alert notification to subscribed users when a flagged article is ingested.	High
FR-35	The system shall send a daily digest notification summarizing the top stories to opted-in users.	Medium
FR-36	The system shall allow a registered user to configure which notification types they wish to receive.	Medium
FR-37	The system shall allow a registered user to disable all notifications.	Medium
FR-38	The system shall log every notification dispatched, including recipient, type, and timestamp, for auditing.	Low

3.6 Bookmark Module

Description and Priority: Allows users to curate a personal reading list, with offline access to saved content. Overall module priority: Medium.

Functional Requirements:

ID	Requirement	Priority
FR-39	The system shall allow a registered user to bookmark an article for later reading.	High
FR-40	The system shall allow a registered user to remove an article from their bookmarks.	High

ID	Requirement	Priority
FR-41	The system shall allow a registered user to view a list of all bookmarked articles.	High
FR-42	The system shall cache the content of a bookmarked article locally to support offline reading.	Medium
FR-43	The system shall notify a user if a bookmarked article becomes unavailable at its original source.	Low

3.7 Feedback Module

Description and Priority: Captures user sentiment about the platform and the AI Agent, and provides a structured channel for bug reports and feature requests. Overall module priority: Low.

Functional Requirements:

ID	Requirement	Priority
FR-44	The system shall allow a user to submit general feedback about the platform.	Medium
FR-45	The system shall allow a user to rate an AI-generated response with a thumbs-up or thumbs-down.	Medium
FR-46	The system shall allow a user to report a bug, including a text description and an optional screenshot.	Medium
FR-47	The system shall allow a user to submit a feature request for review by the product team.	Low

3.8 Admin Module

Description and Priority: Gives administrators the tools required to operate, moderate, and monitor the platform. Overall module priority: High.

Functional Requirements:

ID	Requirement	Priority
FR-48	The system shall allow an administrator to view, disable, or delete a user account.	High
FR-49	The system shall allow an administrator to add or remove an RSS news source.	High
FR-50	The system shall allow an administrator to view platform-wide usage analytics.	Medium
FR-51	The system shall allow an administrator to monitor AI Agent usage, including request volume and token consumption.	Medium
FR-52	The system shall allow an administrator to review and respond to submitted user feedback.	Medium
FR-53	The system shall allow an administrator to generate usage and engagement reports for a specified date range.	Medium
FR-54	The system shall allow an administrator to configure system-wide notification templates.	Low
FR-55	The system shall allow an administrator to grant or revoke administrative privileges for another user account.	High

4. Data Requirements and Database Design

This section defines the logical data model that supports the functional requirements in Section 3. The schema is designed as a normalized relational database, suitable for implementation in any standard RDBMS (e.g., PostgreSQL, MySQL). Each entity below corresponds to a table; primary keys (PK) and foreign keys (FK) are indicated in the Key column.

Users

Field	Type	Key / Constraint	Description
user_id	INT	PK	Unique identifier for the user.
username	VARCHAR(50)	UNIQUE, NOT NULL	Display / login name.
email	VARCHAR(100)	UNIQUE, NOT NULL	Registered email address.
password_hash	VARCHAR(255)	NOT NULL	Salted hash of the user's password.
role	ENUM	NOT NULL	'user' or 'admin'.
preferred_language	VARCHAR(10)		Default translation / UI language.
created_at	DATETIME	NOT NULL	Account creation timestamp.

Articles

Field	Type	Key / Constraint	Description
article_id	INT	PK	Unique identifier for the article.
category_id	INT	FK -> Categories.category_id	Article's assigned category.
title	VARCHAR(255)	NOT NULL	Article headline.
source_url	VARCHAR(500)	NOT NULL	Original BBC RSS article URL.
published_at	DATETIME	NOT NULL	Original publication timestamp.
sentiment	ENUM		AI-computed sentiment: positive / neutral / negative.
is_breaking	BOOLEAN	NOT NULL	Flag used to trigger breaking-news alerts.

Categories

Field	Type	Key / Constraint	Description
category_id	INT	PK	Unique identifier for the category.
name	VARCHAR(50)	UNIQUE, NOT NULL	Category name (e.g., World, Technology).

Bookmarks

Field	Type	Key / Constraint	Description
bookmark_id	INT	PK	Unique identifier for the bookmark record.
user_id	INT	FK -> Users.user_id	Owning user.
article_id	INT	FK -> Articles.article_id	Bookmarked article.
created_at	DATETIME	NOT NULL	Timestamp the bookmark was created.

ReadingHistory

Field	Type	Key / Constraint	Description
history_id	INT	PK	Unique identifier for the history entry.
user_id	INT	FK -> Users.user_id	User who read the article.
article_id	INT	FK -> Articles.article_id	Article that was read.
read_at	DATETIME	NOT NULL	Timestamp the article was opened.

Preferences

Field	Type	Key / Constraint	Description
preference_id	INT	PK	Unique identifier for the preference record.
user_id	INT	FK -> Users.user_id	Owning user.
category_id	INT	FK -> Categories.category_id	Favorited category.

Notifications

Field	Type	Key / Constraint	Description
notification_id	INT	PK	Unique identifier for the notification.
user_id	INT	FK -> Users.user_id	Recipient user.
type	ENUM	NOT NULL	'breaking', 'digest', or 'personalized'.
message	VARCHAR(255)	NOT NULL	Notification content.
sent_at	DATETIME	NOT NULL	Timestamp the notification was dispatched.

AIConversation

Field	Type	Key / Constraint	Description
conversation_id	INT	PK	Unique identifier for the conversation session.
user_id	INT	FK -> Users.user_id	User owning the conversation.
started_at	DATETIME	NOT NULL	Conversation start timestamp.

AIConversationMessage

Field	Type	Key / Constraint	Description
message_id	INT	PK	Unique identifier for the message.
conversation_id	INT	FK -> AIConversation.conversation_id	Parent conversation.
sender	ENUM	NOT NULL	'user' or 'ai'.
content	TEXT	NOT NULL	Message text.
rating	ENUM		'up', 'down', or null.
created_at	DATETIME	NOT NULL	Message timestamp.

Feedback

Field	Type	Key / Constraint	Description
feedback_id	INT	PK	Unique identifier for the feedback entry.
user_id	INT	FK -> Users.user_id	Submitting user.
type	ENUM	NOT NULL	'general', 'bug', or 'feature_request'.
content	TEXT	NOT NULL	Feedback details.
created_at	DATETIME	NOT NULL	Submission timestamp.

TrendingTopics

Field	Type	Key / Constraint	Description
trend_id	INT	PK	Unique identifier for the trend entry.
category_id	INT	FK -> Categories.category_id	Related category.
topic	VARCHAR(100)	NOT NULL	Trending topic label.
read_count	INT	NOT NULL	Aggregate read count within the window.
computed_at	DATETIME	NOT NULL	Timestamp the ranking was computed.

Analytics

Field	Type	Key / Constraint	Description
analytics_id	INT	PK	Unique identifier for the analytics record.
user_id	INT	FK -> Users.user_id	Associated user (nullable for platform-wide records).
metric_name	VARCHAR(100)	NOT NULL	Name of the tracked metric.
metric_value	INT	NOT NULL	Recorded value.
recorded_at	DATETIME	NOT NULL	Timestamp the metric was recorded.

4.1 Entity Relationships

- Users (1) — (M) Bookmarks: a user may bookmark many articles.
- Users (1) — (M) ReadingHistory: a user accumulates many reading-history entries.
- Users (1) — (M) Preferences: a user may favorite many categories.
- Users (1) — (M) Notifications, Feedback, AIConversation, Analytics: each user may generate many records in these tables.
- Categories (1) — (M) Articles: each category groups many articles.
- Categories (1) — (M) Preferences, TrendingTopics: a category may appear in many users' preferences and many trend computations.
- Articles (1) — (M) Bookmarks, ReadingHistory: each article may be bookmarked or read by many users.
- AIConversation (1) — (M) AIConversationMessage: each conversation contains many exchanged messages.

4.2 Normalization

The schema satisfies Third Normal Form (3NF). Each table has a single-column surrogate primary key (1NF: atomic values, no repeating groups). All non-key attributes are fully functionally dependent on the whole primary key rather than a subset of it, which is trivially satisfied because every table uses a single-column key (2NF). Finally, no non-key attribute depends on another non-key attribute — for example, a category's name is stored only in Categories and referenced by category_id elsewhere, rather than being duplicated on every Article row (3NF). This design minimizes update anomalies and storage redundancy while keeping common queries, such as retrieving a user's personalized feed, efficient through indexed foreign keys.

5. External Interface Requirements

5.1 User Interfaces

- The frontend shall provide a responsive layout supporting desktop, tablet, and mobile viewport sizes.
- The frontend shall provide a persistent navigation bar with category links, a search field, and account controls.
- The frontend shall provide a toggle between light and dark themes, persisted per user.
- The frontend shall provide a dedicated AI Chat Assistant panel accessible from any page.
- The frontend shall provide inline controls on each article for summarize, translate, bookmark, and compare actions.

5.2 Hardware Interfaces

- The client requires only a standard personal computer, tablet, or smartphone capable of running a modern web browser; no specialized hardware is required.
- The server requires a host machine with sufficient CPU/GPU resources to run the Ollama-hosted Llama 3.2 model at acceptable latency.

5.3 Software Interfaces

Software	Description
Node.js / Express.js	Backend runtime and REST API framework that serves the frontend and orchestrates business logic.
Relational Database (e.g., PostgreSQL/MySQL)	Persists users, articles, preferences, and all other entities defined in Section 4.
Ollama	Local model-serving runtime that hosts and executes the Llama 3.2 model for all AI features.
BBC RSS Feeds	External source of raw news content, consumed by the News Tool via HTTP.
Web Browser (Chrome, Firefox, Edge, Safari)	Client runtime that renders the HTML/CSS/JavaScript frontend.

5.4 Communication Interfaces

- The frontend and backend shall communicate over HTTPS using a RESTful JSON API.
- The News Tool shall retrieve BBC RSS content over HTTPS at scheduled polling intervals.
- The AI Agent shall communicate with Ollama over a local HTTP interface.
- Real-time notification delivery shall use a push mechanism (e.g., WebSocket or Server-Sent Events) where supported, with polling as a fallback.

6. Non-Functional Requirements

This section specifies the quality attributes the system must exhibit, grouped by category. Each requirement is uniquely identified (NFR-1 through NFR-34) for traceability to test cases.

6.1 Performance

ID	Requirement
NFR-1	The system shall return news search results within 2 seconds for 95% of queries under normal load.
NFR-2	The AI Chat Assistant shall begin streaming a response within 3 seconds under normal load.
NFR-3	The system shall support at least 500 concurrent active users without perceptible degradation.
NFR-4	The personalized news feed shall render within 1.5 seconds of a successful login.
NFR-5	A newly published RSS item shall be ingested and indexed within 60 seconds of its publication.

6.2 Security

ID	Requirement
NFR-6	User passwords shall be stored using a salted, industry-standard hashing algorithm (e.g., bcrypt).
NFR-7	All client-server communication shall be encrypted using TLS 1.2 or higher.
NFR-8	Session tokens shall automatically expire after a configurable period of user inactivity.
NFR-9	The system shall enforce role-based access control, restricting Admin-module functions to Admin accounts.
NFR-10	The system shall validate and sanitize all user input to prevent injection and cross-site scripting attacks.

6.3 Reliability

ID	Requirement
NFR-11	The system shall maintain at least 99.5% uptime measured on a monthly basis.
NFR-12	If the Ollama inference service is unavailable, the system shall degrade gracefully to non-AI news browsing rather than fail.
NFR-13	A failed RSS ingestion job shall be automatically retried up to three times before being logged as failed.

6.4 Availability

ID	Requirement
NFR-14	The system shall be available for use 24 hours a day, 7 days a week, excluding scheduled maintenance.
NFR-15	Scheduled maintenance windows shall be announced to users at least 24 hours in advance.
NFR-16	Backend service updates shall be deployable with zero downtime.

6.5 Scalability

ID	Requirement
NFR-17	The backend shall be capable of horizontal scaling to accommodate growth in concurrent user load.
NFR-18	The database shall support partitioning of high-growth tables such as ReadingHistory and AIConversation.
NFR-19	The AI Agent layer shall support pluggable model backends to allow inference capacity to scale independently.

6.6 Maintainability

ID	Requirement
NFR-20	The codebase shall follow a modular, layered architecture that isolates changes to individual layers.
NFR-21	Backend modules shall maintain a minimum of 70% automated test coverage.
NFR-22	Each functional module shall expose a clearly defined service boundary to support independent maintenance.

6.7 Portability

ID	Requirement
NFR-23	The frontend shall render and function correctly on the latest two major versions of Chrome, Firefox, Edge, and Safari.
NFR-24	The backend shall be deployable in any containerized, Node.js-compatible hosting environment.

6.8 Accessibility

ID	Requirement
NFR-25	The frontend shall conform to WCAG 2.1 Level AA accessibility guidelines.
NFR-26	Article content shall be available through screen-reader compatible text-to-speech playback.

6.9 Privacy

ID	Requirement
NFR-27	The system shall allow a user to request export or deletion of their personal data.
NFR-28	The system shall not share a user's reading history or personal data with third parties without explicit consent.

6.10 Backup & Recovery

ID	Requirement
NFR-29	The production database shall be backed up daily with a minimum 30-day retention period.
NFR-30	The system shall support point-in-time recovery, restoring service within 24 hours of a data-loss incident.

6.11 Logging

ID	Requirement
NFR-31	The system shall log all authentication events, administrative actions, and AI Agent invocations.
NFR-32	Application logs shall be retained for a minimum of 90 days.

6.12 Monitoring

ID	Requirement
NFR-33	The system shall provide a real-time monitoring dashboard showing API latency and error rates.
NFR-34	The system shall alert administrators automatically when the AI Agent error rate exceeds a defined threshold.

7. UML Modeling Support

This section consolidates the actors, classes, entities, states, and interactions described elsewhere in this document so that the diagrams listed below can be produced directly from this specification.

7.1 Actors

Actor	Type	Description
Guest User	Primary (human)	Unauthenticated visitor browsing public content.
Registered User	Primary (human)	Authenticated end user with personalization, AI, and social features.
Administrator	Primary (human)	Internal user managing accounts, content sources, and analytics.
AI Agent	Secondary (system)	Internal system actor composed of the Planner, Executor, and Memory that services AI feature requests.
News Tool	Secondary (system)	Internal system actor that retrieves and filters articles from BBC RSS.
Ollama (Llama 3.2)	External system	External inference service invoked by the AI Agent.
Notification Service	Secondary (system)	Internal system actor responsible for dispatching alerts and digests.

7.2 Use Case Diagram

A Use Case Diagram may be constructed using the actors in Section 7.1 and the complete use case catalog in Section 8. Guest User connects to browsing- and search-related use cases; Registered User connects to all use cases except administrative ones; Administrator connects to the management use cases (UC-22 through UC-24); the AI Agent and News Tool appear as supporting actors on AI-related use cases via include relationships.

7.3 Class Diagram

Candidate classes correspond to the entities in Section 4: User, Article, Category, Bookmark, ReadingHistory, Preference, Notification, AIConversation, AIConversationMessage, Feedback, TrendingTopic, and Analytics. Service-layer classes include AuthService, NewsService, RecommendationService, AIAgentService (composed of Planner, Executor, and MemoryStore), NotificationService, BookmarkService, FeedbackService, and AdminService. Associations follow the relationships enumerated in Section 4.1.

7.4 Entity-Relationship (ER) Diagram

The ER Diagram is derived directly from Section 4: twelve entities connected by the one-to-many relationships listed in Section 4.1, with primary and foreign keys as specified in each entity's field table.

7.5 Sequence Diagrams

Representative sequence diagrams can be constructed from the Main Flow steps of the use cases in Section 8, most notably UC-13 (Chat with AI Assistant), which traces the interaction User → Frontend → Express API → AI Agent → Planner → Executor → News Tool / Ollama → Response, and UC-01 (Register Account), which traces User → Frontend → Express API → AuthService → Users table.

7.6 Activity Diagram

An activity diagram for the personalized-feed generation process flows as follows: Start → Retrieve user preferences → Retrieve reading history → Query candidate articles → Decision: does the AI Agent recommend additional articles? → Merge and rank results → Render feed → End. A second activity diagram can be built from the RSS ingestion pipeline: Poll BBC RSS → Parse new items → Classify topic and sentiment via AI → Flag breaking news → Persist Article → Trigger notifications if flagged → End.

7.7 State Diagram

A state diagram for an `AIConversationMessage` transitions through the states: Submitted → Planning → Executing → Awaiting Model Response → Delivered → Rated (optional, terminal). A state diagram for a Notification transitions through: Queued → Sent → Delivered, with a Failed state reachable from Sent after exhausting retry attempts (see NFR-13).

7.8 Component Diagram

Major components are: Frontend (Web UI), Express API Gateway, Authentication Service, News Service, Recommendation Service, AI Agent (Planner, Executor, Memory), News Tool, Notification Service, Admin Service, and the Database. Ollama is modeled as an external component consumed by the AI Agent.

7.9 Deployment Diagram

The deployment view consists of a Client Node (web browser) communicating over HTTPS with an Application Server Node (Node.js/Express, hosting the AI Agent and all backend services), which in turn communicates with a Database Server Node and a locally deployed Model Server Node running Ollama and the Llama 3.2 model.

7.10 Package Diagram

The backend codebase is organized into packages: auth, news, personalization, ai-agent (with sub-packages planner, executor, memory, tools), trending, notifications, bookmarks, feedback, and admin, each depending on a shared data package that exposes the repository layer for the entities in Section 4.

8. Use Cases

The following use cases elaborate the functional requirements of Section 3 into complete user interactions, including alternative flows and exception handling. They form the basis for functional test-case design.

UC-01: Register Account	
Actors	Guest User
Preconditions	The user has a valid email address and is not already registered.
Main Flow	1. User navigates to the registration page. 2. User enters email, username, and password. 3. System validates that the email is unique and the password meets complexity rules. 4. System creates the account and sends a confirmation email. 5. System logs the user in and redirects to the personalized feed setup.
Alternative Flow	If the email is already registered, the system displays an error and offers a login link instead.
Exceptions	If the registration service is unavailable, the system displays a retry message and preserves entered data.
Postconditions	A new user account exists in the Users table and the user is authenticated.

UC-02: Log In	
Actors	Registered User
Preconditions	The user holds a registered account.
Main Flow	1. User enters email/username and password. 2. System validates the credentials. 3. System issues a session token. 4. System redirects the user to their personalized feed.
Alternative Flow	If “Remember Me” is selected, the system issues a longer-lived refresh token.
Exceptions	If credentials are invalid, the system displays an error and increments a failed-attempt counter.
Postconditions	The user holds an active authenticated session.

UC-03: Log Out	
Actors	Registered User
Preconditions	The user is authenticated.
Main Flow	1. User selects “Log Out.” 2. System invalidates the active session token. 3. System redirects the user to the public homepage.
Alternative Flow	N/A
Exceptions	If the token is already expired, the system simply redirects the user without error.
Postconditions	The user's session token is invalidated.

UC-04: Reset Password	
Actors	Registered User
Preconditions	The user has a registered email address.
Main Flow	1. User requests a password reset from the login page. 2. System emails a time-limited reset link. 3. User opens the link and submits a new password. 4. System validates and updates the stored password hash.
Alternative Flow	If the user requests multiple resets, only the most recent link remains valid.
Exceptions	If the reset link has expired, the system prompts the user to request a new one.
Postconditions	The user's password is updated and prior sessions are invalidated.

UC-05: Update Profile	
Actors	Registered User
Preconditions	The user is authenticated.
Main Flow	1. User navigates to profile settings. 2. User edits display name, avatar, or preferred language. 3. System validates and saves the changes.
Alternative Flow	N/A
Exceptions	If an uploaded avatar exceeds the size limit, the system rejects it with an explanatory message.
Postconditions	The user's profile record is updated.

UC-06: Browse News by Category	
Actors	Guest User, Registered User
Preconditions	None.
Main Flow	1. User selects a category from the navigation bar. 2. System retrieves and displays articles belonging to that category. 3. User scrolls to load additional articles.
Alternative Flow	If no articles exist for the category, the system displays an empty-state message.
Exceptions	If the news source is temporarily unavailable, the system displays cached articles with a notice.
Postconditions	The requested category's articles are displayed.

UC-07: Search News	
Actors	Guest User, Registered User

Preconditions	None.
Main Flow	1. User enters a search term. 2. System queries indexed articles for matches. 3. System displays ranked search results.
Alternative Flow	User applies a category or date filter to narrow results.
Exceptions	If the search term returns no matches, the system suggests related trending topics.
Postconditions	Matching articles are displayed to the user.

UC-08: Bookmark an Article

Actors	Registered User
Preconditions	The user is authenticated and viewing an article.
Main Flow	1. User selects the “Bookmark” action on an article. 2. System creates a Bookmarks record linking the user and article. 3. System confirms the article was saved.
Alternative Flow	N/A
Exceptions	If the article is already bookmarked, the system disables the action and shows it as saved.
Postconditions	The article appears in the user's bookmark list.

UC-09: Remove a Bookmark

Actors	Registered User
Preconditions	The user has at least one bookmarked article.
Main Flow	1. User opens their bookmark list. 2. User selects “Remove” on an article. 3. System deletes the corresponding Bookmarks record.
Alternative Flow	N/A
Exceptions	N/A
Postconditions	The article no longer appears in the user's bookmark list.

UC-10: View Reading History

Actors	Registered User
Preconditions	The user is authenticated.
Main Flow	1. User navigates to “Reading History.” 2. System retrieves and displays articles the user has previously opened, most recent first.
Alternative Flow	User clears their reading history from this screen.
Exceptions	N/A

Postconditions	The user's reading history is displayed.
-----------------------	--

UC-11: Set Favorite Categories

Actors	Registered User
Preconditions	The user is authenticated.
Main Flow	1. User opens category preferences. 2. User selects or deselects categories of interest. 3. System saves the updated Preferences record.
Alternative Flow	N/A
Exceptions	N/A
Postconditions	The user's favorite categories are updated and the personalized feed is regenerated.

UC-12: View Personalized News Feed

Actors	Registered User
Preconditions	The user is authenticated and has at least one preference or reading history entry.
Main Flow	1. User opens the home feed. 2. System ranks articles using favorite categories, reading history, and AI recommendations. 3. System displays the ranked feed.
Alternative Flow	If the user has no preferences yet, the system displays a general trending feed instead.
Exceptions	N/A
Postconditions	A personalized feed is displayed to the user.

UC-13: Chat with AI Assistant

Actors	Registered User, AI Agent
Preconditions	The user is authenticated.
Main Flow	1. User opens the AI Chat Assistant and types a question. 2. The AI Agent's Planner decomposes the request into a plan. 3. The Executor invokes the News Tool as needed and queries Ollama for a response. 4. System streams the AI response back to the user.
Alternative Flow	User asks a follow-up question; the AI Agent uses session memory for context.
Exceptions	If Ollama is unavailable, the system informs the user that AI features are temporarily offline.
Postconditions	An AI response is displayed and logged to the AIConversation record.

UC-14: Generate AI Daily Brief

Actors	Registered User, AI Agent
Preconditions	The user is authenticated and has reading history or preferences.
Main Flow	1. System triggers the daily brief job for the user. 2. The AI Agent selects and summarizes top relevant stories. 3. System delivers the brief to the user's feed and, if enabled, via notification.
Alternative Flow	N/A
Exceptions	If insufficient articles are available, the system generates a brief from general trending stories.
Postconditions	A daily brief is generated and stored for the user.

UC-15: Summarize an Article with AI

Actors	Registered User, AI Agent
Preconditions	The user is viewing an article.
Main Flow	1. User selects "Summarize." 2. The AI Agent requests a summary from Ollama. 3. System displays the generated summary beneath the article.
Alternative Flow	N/A
Exceptions	If summarization fails, the system displays an error and allows a retry.
Postconditions	A summary is displayed and may be cached for future requests.

UC-16: Translate an Article

Actors	Registered User, AI Agent
Preconditions	The user is viewing an article.
Main Flow	1. User selects a target language. 2. The AI Agent requests a translation from Ollama. 3. System displays the translated article.
Alternative Flow	N/A
Exceptions	If the target language is unsupported, the system notifies the user.
Postconditions	A translated version of the article is displayed.

UC-17: Compare Multiple News Articles

Actors	Registered User, AI Agent
Preconditions	The user has selected two or more articles.
Main Flow	1. User selects "Compare" on the chosen articles. 2. The AI Agent analyzes each article and generates a comparison. 3. System displays similarities, differences, and differing perspectives.

Alternative Flow	N/A
Exceptions	If fewer than two articles are selected, the system prompts the user to select more.
Postconditions	A comparison result is displayed to the user.

UC-18: View Trending Topics

Actors	Guest User, Registered User
Preconditions	None.
Main Flow	1. User opens the “Trending” section. 2. System displays currently trending topics and popular articles.
Alternative Flow	N/A
Exceptions	N/A
Postconditions	Trending topics are displayed.

UC-19: Receive Breaking News Notification

Actors	Registered User, Notification Service
Preconditions	The user has enabled breaking news notifications.
Main Flow	1. A breaking-news article is ingested and flagged. 2. The Notification Service dispatches an alert to subscribed users. 3. User receives and opens the notification.
Alternative Flow	N/A
Exceptions	If delivery fails, the system logs the failure and retries once.
Postconditions	The notification is recorded in the Notifications table.

UC-20: Submit Feedback

Actors	Registered User
Preconditions	The user is authenticated.
Main Flow	1. User opens the feedback form. 2. User enters feedback text and submits. 3. System stores the feedback and confirms receipt.
Alternative Flow	N/A
Exceptions	N/A
Postconditions	A Feedback record is created.

UC-21: Rate an AI Response

Actors	Registered User
Preconditions	The user has received an AI-generated response.
Main Flow	1. User selects thumbs-up or thumbs-down on the response. 2. System stores the rating linked to the AIConversation record.
Alternative Flow	N/A
Exceptions	N/A
Postconditions	The AI response rating is stored for quality monitoring.

UC-22: Manage Users

Actors	Admin
Preconditions	The admin is authenticated with administrative privileges.
Main Flow	1. Admin opens the user management dashboard. 2. Admin searches for a user account. 3. Admin views, disables, or deletes the account as required.
Alternative Flow	N/A
Exceptions	The system prevents an admin from deleting their own account through this workflow.
Postconditions	The targeted user account's status is updated.

UC-23: Manage News Sources

Actors	Admin
Preconditions	The admin is authenticated with administrative privileges.
Main Flow	1. Admin opens the news source management screen. 2. Admin adds a new RSS feed URL or removes an existing one. 3. System validates and saves the change.
Alternative Flow	N/A
Exceptions	If the RSS URL is invalid or unreachable, the system rejects the change with an error.
Postconditions	The list of active news sources is updated.

UC-24: View Platform Analytics

Actors	Admin
Preconditions	The admin is authenticated with administrative privileges.
Main Flow	1. Admin opens the analytics dashboard. 2. System displays usage metrics, AI usage statistics, and engagement trends. 3. Admin optionally generates a report for a selected date range.
Alternative Flow	N/A

Exceptions	N/A
Postconditions	Analytics data is displayed or exported as a report.

9. User Stories

The user stories below express the same scope as Sections 3 and 8 from the perspective of the end user, in the standard “As a ... I want ... so that ...” format, and may be used directly to seed a product backlog.

#	User Story
1	As a guest visitor, I want to browse news by category, so that I can quickly find topics I care about.
2	As a guest visitor, I want to search for news articles, so that I can find coverage of a specific event.
3	As a guest visitor, I want to register for an account, so that I can personalize my news experience.
4	As a registered user, I want to log in securely, so that I can access my personalized feed.
5	As a registered user, I want to reset my password, so that I can regain access if I forget it.
6	As a registered user, I want to update my profile, so that my account reflects my current preferences.
7	As a registered user, I want to select my favorite categories, so that my feed matches my interests.
8	As a registered user, I want the system to remember articles I've read, so that I don't lose track of my reading.
9	As a registered user, I want a personalized news feed, so that I see the most relevant stories first.
10	As a registered user, I want AI-powered article recommendations, so that I discover related stories I might have missed.
11	As a registered user, I want to bookmark an article, so that I can read it later.
12	As a registered user, I want to remove a bookmark, so that my saved list stays relevant.
13	As a registered user, I want offline access to bookmarked articles, so that I can read without an internet connection.
14	As a registered user, I want to chat with an AI assistant about the news, so that I can get quick, conversational answers.
15	As a registered user, I want an AI-generated daily brief, so that I can catch up on important stories in minutes.
16	As a registered user, I want an AI summary of a long article, so that I can grasp the key points quickly.
17	As a registered user, I want articles translated into my preferred language, so that I can read news regardless of the source language.
18	As a registered user, I want a simplified explanation of complex news, so that I can understand unfamiliar topics.
19	As a registered user, I want to compare multiple articles on the same event, so that I can see different perspectives.
20	As a registered user, I want to see the sentiment of an article, so that I can gauge its tone before reading.
21	As a registered user, I want to view trending topics, so that I stay aware of what others are reading.
22	As a registered user, I want to view my reading analytics, so that I can understand my own news habits.

#	User Story
23	As a registered user, I want to receive breaking news alerts, so that I don't miss urgent stories.
24	As a registered user, I want to receive a daily digest notification, so that I get a consistent news summary.
25	As a registered user, I want to control which notifications I receive, so that I'm not overwhelmed.
26	As a registered user, I want to submit feedback about the platform, so that I can help improve it.
27	As a registered user, I want to rate AI responses, so that the assistant's quality can be monitored and improved.
28	As a registered user, I want to report a bug with a screenshot, so that issues are easier for the team to diagnose.
29	As a registered user, I want dark mode, so that I can read comfortably at night.
30	As an administrator, I want to manage user accounts, so that I can maintain platform integrity.
31	As an administrator, I want to manage RSS news sources, so that the platform's content stays current and reliable.
32	As an administrator, I want to view platform analytics, so that I can make informed product decisions.
33	As an administrator, I want to monitor AI Agent usage, so that I can control operational costs and detect misuse.
34	As an administrator, I want to review user feedback, so that I can prioritize improvements.

10. Operating Environment

10.1 Client Environment

- Any device (desktop, laptop, tablet, or smartphone) capable of running a modern web browser.
- Minimum screen width of 320px supported through the responsive frontend layout.

10.2 Server Environment

- Node.js LTS runtime executing the Express.js backend.
- A relational database server (e.g., PostgreSQL or MySQL) reachable by the backend.
- A machine capable of hosting Ollama and serving the Llama 3.2 model, with GPU acceleration recommended for acceptable inference latency.

10.3 Browser Support

- The latest two major versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.

10.4 Operating Systems

- Server: any Linux distribution, Windows Server, or macOS capable of running Node.js and Ollama.
- Client: any operating system with a supported browser, including Windows, macOS, Linux, iOS, and Android.

10.5 Network Requirements

- A stable internet connection is required on the client for all network-dependent features; offline reading is supported only for previously cached bookmarks.
- The server requires outbound HTTPS access to the BBC RSS feed endpoints.

10.6 Deployment Environment

- The application is designed for containerized deployment (e.g., Docker) to simplify environment parity between development, testing, and the course demonstration environment.

11. Software Architecture

World In Brief follows a layered architecture that cleanly separates presentation, orchestration, AI reasoning, and persistence concerns. Requests flow downward from the Presentation Layer to the Database and Ollama, while responses flow back upward through the same layers. This separation allows the AI Agent Layer to be modified, extended with new tools, or even swapped for a different model backend (see NFR-19) without impacting the Presentation or Repository layers.

Layer	Responsibility
Presentation Layer	Renders the responsive HTML/CSS/JavaScript user interface, including dark mode, search, and category navigation, and captures user input.
API Layer	Exposes RESTful endpoints via Express.js, validates incoming requests, and routes them to the appropriate service.
Authentication Layer	Verifies credentials, issues and validates JWT session tokens, and enforces role-based access control.
Business Logic Layer	Implements the core rules for personalization, trending computation, bookmarking, notifications, and feedback handling.
AI Agent Layer	Coordinates AI feature requests, delegating to the Planner and Executor and maintaining short-term conversational memory.
Planner	Interprets a user's natural-language request and decomposes it into an ordered sequence of executable actions.
Executor	Carries out each planned action, invoking the News Tool or the underlying model as required, and assembles the final result.
Service Layer	Provides reusable domain services (NewsService, RecommendationService, NotificationService, etc.) consumed by both the Business Logic Layer and the AI Agent Layer.
Repository Layer	Abstracts data access, translating service-layer calls into queries against the relational database.
Database	Persists all entities defined in Section 4 in a normalized relational schema.
Ollama	Hosts and executes the Llama 3.2 model, returning inference results to the Executor.

12. Future Scope

The following enhancements are recognized as valuable extensions to the platform but are out of scope for the current release:

- Persistent, long-term AI memory that recalls user preferences and past conversations across sessions, beyond the current session-scoped memory.
- Voice-based interaction with the AI Chat Assistant, in addition to the existing text-to-speech playback of articles.
- A native mobile application for iOS and Android, complementing the responsive web frontend.
- AI-driven fact-checking that cross-references claims in an article against multiple independent sources.
- Multi-source news aggregation beyond BBC RSS, to broaden coverage and enable cross-source comparison.
- A real-time trending dashboard with live-updating visualizations for administrators and power users.
- On-demand PDF report generation for reading analytics and admin usage reports.
- AI-generated news podcast summaries, converting the AI Daily Brief into an audio format.

WORLD IN BRIEF

AI News Intelligence Platform

Work Breakdown Structure (WBS) Gantt Chart & Project Schedule

Project Duration: 3 Months (12 Weeks)

Derived from the Software Requirements Specification (IEEE 830-1998)

Prepared for Academic Submission — Software Engineering Course Project

Version 1.0

1. What is a Work Breakdown Structure?

Breaking work into smaller tasks is a common productivity technique used to make work more manageable and approachable. For projects, the Work Breakdown Structure (WBS) is the tool that applies this technique and is one of the most important project management documents. It singlehandedly integrates the scope, cost, and schedule baselines, ensuring that project plans stay in alignment.

The Project Management Institute (PMI), in the Project Management Book of Knowledge (PMBOK), defines the Work Breakdown Structure as a “deliverable-oriented hierarchical decomposition of the work to be executed by the project team.” There are two common types of WBS: (1) Deliverable-Based and (2) Phase-Based. The Deliverable-Based approach is the most common and preferred approach. The main difference between the two lies in how the Level 1 elements are defined.

1.1 Deliverable-Based WBS

A Deliverable-Based WBS clearly demonstrates the relationship between the project's deliverables (products, services, or results) and its scope (the work to be executed). For World In Brief, the Level 1 elements are the major system deliverables — the Authentication System, the Personalized News System, the AI Intelligence System, and so on — each traceable directly to a module in Section 3 of the SRS.

1.2 Phase-Based WBS

In a Phase-Based WBS, the Level 1 elements are the typical phases of the project (e.g., Initiation, Design, Development, Testing, Deployment), and the Level 2 elements are the unique deliverables produced within each phase. Because the lower-level elements are still deliverables regardless of the WBS type, elements belonging to different phases can share similar names; a WBS Dictionary is used to describe precisely what work is unique to each element. A Phase-Based WBS maps naturally onto a project timeline, which makes it the structure used to build the Gantt chart for World In Brief in Section 6 of this document.

A good WBS is simply one that makes the project more manageable. Every project, project manager, and WBS is different — the right WBS is whichever structure best answers the question: “What structure makes this project more manageable?”

2. How to Make a Work Breakdown Structure

A good Work Breakdown Structure is created through an iterative process, following these steps and guidelines:

Step 1 — Gather Critical Documents

- Gather critical project documents — for World In Brief, this is the Software Requirements Specification (SRS).
- Identify content describing project deliverables, such as the scope statement in SRS Section 1.4 and the feature list in Section 2.2.

Step 2 — Identify Key Team Members

- Identify the appropriate project team members (frontend, backend, AI, QA, and DevOps roles).
- Analyze the SRS and identify the deliverables it implies.

Step 3 — Define Level 1 Elements

- Define the Level 1 elements as summary deliverable descriptions that together capture 100% of the project scope.
- Verify that 100% of scope is captured — this is commonly referred to as the 100% Rule.

Step 4 — Decompose (Break Down) Elements

- Break each Level 1 deliverable into unique, lower-level deliverables (Level 2 work packages).
- Continue decomposing until each element can be managed by a single individual or sub-team, and ensure elements remain mutually exclusive.

Step 5 — Create the WBS Dictionary

- Define the content of the WBS Dictionary — a narrative description of the work covered by each element. The lowest-level elements are called Work Packages.
- Write dictionary descriptions at the Work Package level with enough detail to ensure 100% of scope is covered, including owner, duration, and boundaries.

Step 6 — Create the Gantt Chart Schedule

- Decompose Work Packages into activities where appropriate.
- Enter the WBS into a Gantt chart for scheduling and tracking — shown for World In Brief in Section 6.

3. World In Brief — Work Breakdown Structure

The two diagrams below apply the theory in Section 1 directly to World In Brief. Figure 1 presents a Phase-Based WBS — the structure used to build the 3-month Gantt chart in Section 6. Figure 2 presents a Deliverable-Based WBS, mapping each Level 1 element directly onto a functional module of the SRS (Section 3).

3.1 Phase-Based WBS (used for scheduling)

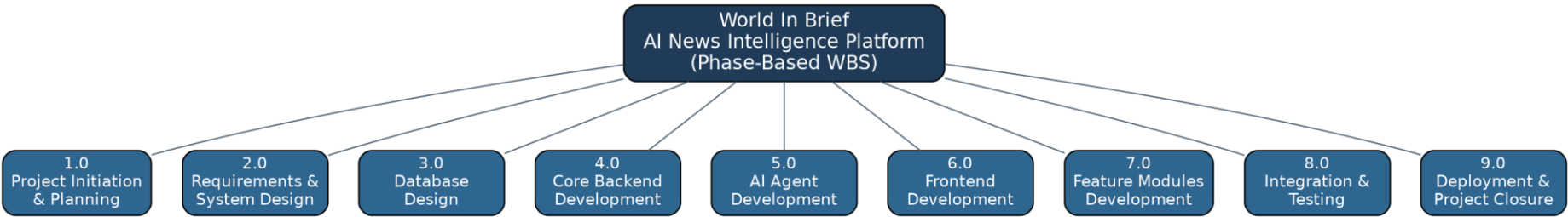


Figure 1 — Phase-Based Work Breakdown Structure for World In Brief (Level 0–1)

3.2 Deliverable-Based WBS

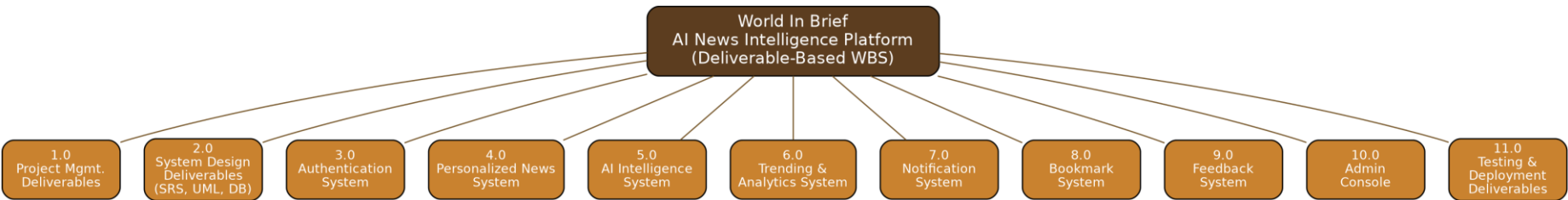


Figure 2 — Deliverable-Based Work Breakdown Structure for World In Brief (Level 0–1)

4. Outlined WBS

The diagram below expands the Phase-Based WBS in Figure 1 down to its full four-level hierarchy — Project → Task → Subtask → Work Package — in the same nested, outlined-box format shown in Figure 3 of the WBS template. Each Task corresponds to a Level 1 phase, each Subtask to a Level 2 element from the WBS Dictionary (Section 5), and each Work Package to a concrete, assignable activity. It is shown across three parts (Phases 1–3, 4–6, and 7–9) so the boxes remain legible.

Project Name	Task	Subtask	Work Package
World In Brief	1.0 Project Initiation & Planning	1.1 Kickoff & Scope Definition	1.1.1 Conduct project kickoff meeting
			1.1.2 Confirm scope against SRS Section 1.4
		1.2 Team & Tool Setup	1.2.1 Set up dev/staging environments
			1.2.2 Configure Git repo & task board
		1.3 Project Plan, WBS & Schedule	1.3.1 Build WBS & WBS Dictionary
			1.3.2 Build 12-week Gantt schedule
	2.0 Requirements & System Design	2.1 SRS Finalization	2.1.1 Review & lock FR-1..55
			2.1.2 Review & lock NFR-1..34
		2.2 UML Modeling	2.2.1 Use Case & Class diagrams
			2.2.2 Sequence, Activity & State diagrams
		2.3 System Architecture Design	2.3.1 Define layered architecture
			2.3.2 Define AI Agent architecture
	3.0 Database Design	3.1 ER Diagram & Schema Design	3.1.1 Model the 12 entities
			3.1.2 Define relationships & keys
		3.2 Normalization & DB Setup	3.2.1 Verify 3NF compliance
			3.2.2 Provision DB instance & migrations

Figure 3a — Outlined WBS for World In Brief, Part 1 of 3 (Phases 1.0–3.0)

Project Name	Task	Subtask	Work Package
World In Brief	4.0 Core Backend Development	4.1 Authentication Module	4.1.1 Register / Login / Logout
			4.1.2 JWT issuance & password reset
		4.2 Personalized News Module	4.2.1 Favorites & reading history
			4.2.2 Personalized feed ranking
		4.3 REST API & Express Backend	4.3.1 Routing & request validation
			4.3.2 RBAC middleware
	5.0 AI Agent Development	5.1 Ollama & Llama 3.2 Setup	5.1.1 Install & configure Ollama
			5.1.2 Model performance tuning
		5.2 Planner & Executor Components	5.2.1 Build the Planner
			5.2.2 Build the Executor & tool invocation
		5.3 AI Features	5.3.1 Summarize / Translate / Classify
			5.3.2 Sentiment / Headline / Explain / Compare
	6.0 Frontend Development	6.1 UI/UX & Responsive Layout	6.1.1 Nav bar & theme toggle
			6.1.2 Responsive grid system
		6.2 Feed & Article Views	6.2.1 Category browsing & search
			6.2.2 Article detail view
		6.3 AI Chat Assistant Interface	6.3.1 Chat panel UI
			6.3.2 Streaming response handling

Figure 3b — Outlined WBS for World In Brief, Part 2 of 3 (Phases 4.0–6.0)

Project Name	Task	Subtask	Work Package
World In Brief	7.0 Feature Modules Development	7.1 Trending Module	7.1.1 Trending computation job
			7.1.2 Trending UI display
		7.2 Notification Module	7.2.1 Breaking-news alerts
			7.2.2 Digest & preferences
		7.3 Bookmark Module	7.3.1 Bookmark CRUD
			7.3.2 Offline caching
		7.4 Feedback & Admin Module	7.4.1 Feedback / rating / bug report
			7.4.2 Admin console
	8.0 Integration & Testing	8.1 Module Integration	8.1.1 End-to-end wiring
			8.1.2 Interface / data-contract fixes
		8.2 Unit & Functional Testing	8.2.1 Unit test suite
			8.2.2 Functional / use-case tests
		8.3 Performance, Security & UAT	8.3.1 Load & latency testing
			8.3.2 Security testing & UAT
	9.0 Deployment & Project Closure	9.1 Deployment & Documentation	9.1.1 Containerized deployment
			9.1.2 User & technical documentation
		9.2 Final Demo & Project Closure	9.2.1 Final demonstration
			9.2.2 Closure report & handover

Figure 3c — Outlined WBS for World In Brief, Part 3 of 3 (Phases 7.0–9.0)

5. WBS Dictionary

Following the guidance in Section 2, the table below is the WBS Dictionary for the Phase-Based WBS in Figure 1. It decomposes each Level 1 phase into its Level 2 Work Packages, and for each Work Package records a description, the responsible owner, and its planned duration — the same data used to build the Gantt chart in Section 6. Work Package IDs trace to the numbering in Figure 1 (e.g., 4.1, 4.2, 4.3 belong to Phase 4.0 — Core Backend Development).

WBS ID	Work Package	Description	Owner	Duration
1.1	Kickoff & Scope Definition	Hold project kickoff, confirm scope per SRS Section 1.4, agree on objectives and constraints.	Project Team	0.5 wk
1.2	Team & Tool Setup	Set up Node.js/Express, database, Ollama, Git repo, task board, and dev environments.	Project Team	0.5 wk
1.3	Project Plan, WBS & Schedule	Produce WBS, WBS dictionary, and Gantt schedule for the 3-month timeline.	Project Lead	1 wk
2.1	SRS Finalization	Review and lock the SRS (FR-1..55, NFR-1..34) as the requirements baseline.	Business Analyst	1 wk
2.2	UML Modeling	Produce Use Case, Class, Sequence, Activity, State, Component and Deployment diagrams (SRS Sec. 7).	System Designer	1.5 wk
2.3	System Architecture Design	Finalize the layered architecture: Presentation, API, Auth, Business Logic, AI Agent, Repository (SRS Sec. 11).	System Architect	1 wk
3.1	ER Diagram & Schema Design	Design the 12-entity relational schema and relationships defined in SRS Section 4.	DB Designer	1 wk
3.2	Normalization & DB Setup	Verify 3NF compliance, create database instance, indexes, and migration scripts.	DB Designer	1 wk
4.1	Authentication Module	Implement register, login, logout, JWT issuance, password reset (FR-1..8).	Backend Dev	1.5 wk
4.2	Personalized News Module	Implement favorites, reading history, personalized feed ranking (FR-9..16).	Backend Dev	1.5 wk
4.3	REST API & Express Backend	Build the Express REST API layer, routing, validation, and RBAC middleware.	Backend Dev	3 wk
5.1	Ollama & Llama 3.2 Setup	Install and configure local Ollama runtime serving the Llama 3.2 model.	AI Engineer	1 wk
5.2	Planner & Executor Components	Build the Planner (request decomposition) and Executor (tool invocation) of the AI Agent (FR-17..19).	AI Engineer	1.5 wk
5.3	AI Features	Implement summarization, translation, classification, sentiment, headline, explain, compare (FR-20..27).	AI Engineer	1.5 wk
6.1	UI/UX & Responsive Layout	Build responsive HTML/CSS/JS layout, navigation bar, light/dark theme toggle.	Frontend Dev	1.5 wk
6.2	Feed & Article Views	Build category browsing, search, personalized feed, and article detail views.	Frontend Dev	1.5 wk
6.3	AI Chat Assistant Interface	Build the persistent AI Chat panel with streaming responses (FR-17, NFR-2).	Frontend Dev	1.5 wk
7.1	Trending Module	Implement trending topic computation and popular-article display (FR-29..33).	Backend Dev	1 wk
7.2	Notification Module	Implement breaking-news and digest notifications with user preferences (FR-34..38).	Backend Dev	1.5 wk
7.3	Bookmark Module	Implement bookmarking, removal, listing, and offline caching (FR-39..43).	Backend Dev	1 wk
7.4	Feedback & Admin Module	Implement feedback/rating/bug-report and the admin console (FR-44..55).	Full-Stack Dev	1.5 wk
8.1	Module Integration	Integrate all modules end-to-end; resolve interface and data-contract issues.	Project Team	1.5 wk
8.2	Unit & Functional Testing	Execute unit and functional tests against the use cases in SRS Section 8.	QA Engineer	1 wk
8.3	Performance, Security & UAT	Validate NFR-1..34 (load, latency, security) and run user acceptance testing.	QA Engineer	1.5 wk

WBS ID	Work Package	Description	Owner	Duration
9.1	Deployment & Documentation	Deploy to a containerized environment; finalize user and technical documentation.	DevOps / Team	1.5 wk
9.2	Final Demo & Project Closure	Conduct final demonstration, submit deliverables, and close out the project.	Project Team	1 wk

6. Gantt Chart — 3-Month Project Schedule

The Gantt chart below schedules every Work Package from the WBS Dictionary across a 12-week (3-month) timeline, grouped and color-coded by WBS Level 1 phase. Bars that overlap (for example, 4.3 REST API & Express Backend running alongside 4.1 and 4.2) indicate parallel work by different sub-teams; the schedule assumes a small, cross-functional team of roughly 4–6 members (backend, AI, frontend, QA/DevOps) working concurrently once Phase 4 begins.

World In Brief — 3-Month Project Gantt Chart (12 Weeks)

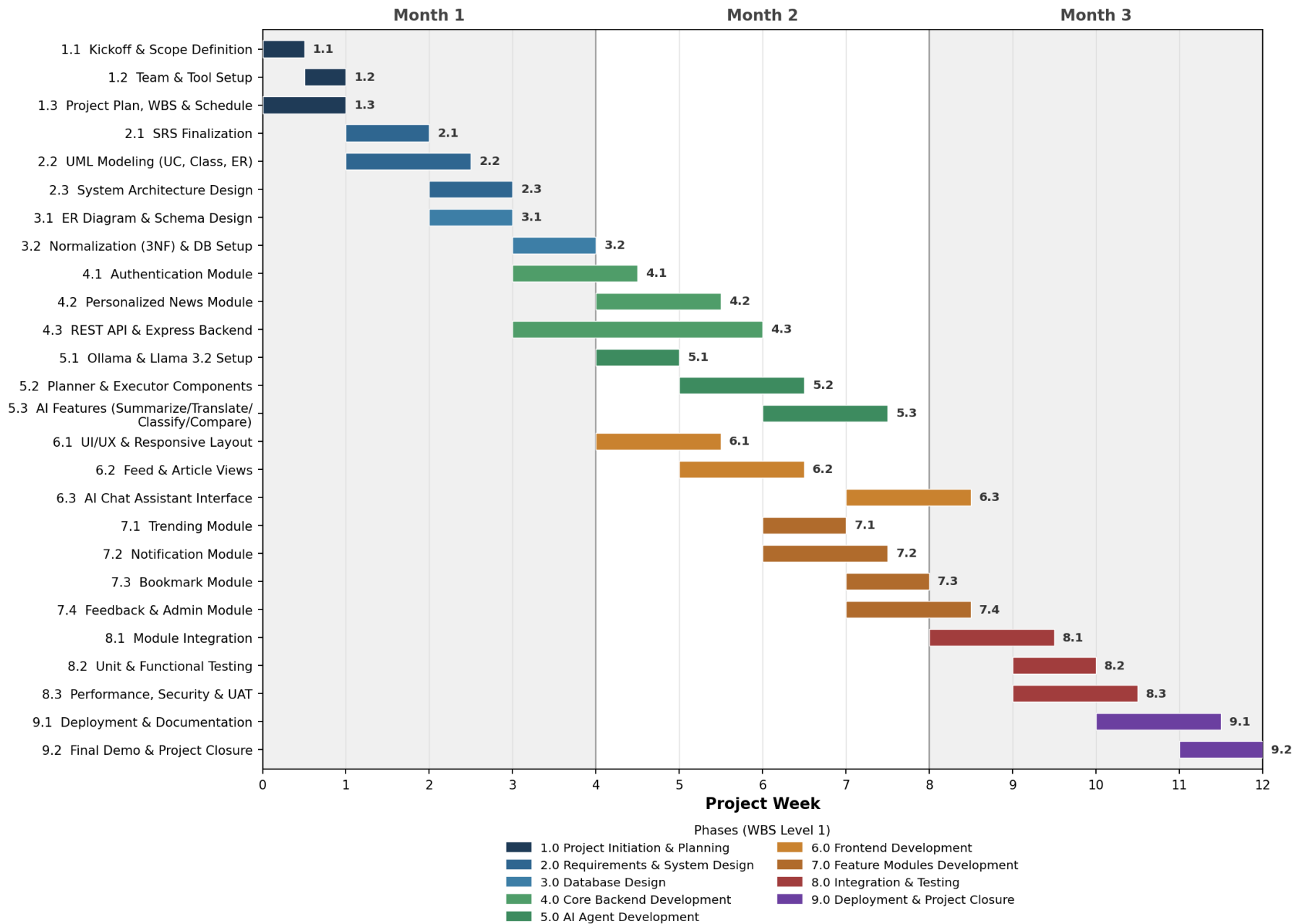


Figure 4 — 3-Month Gantt Chart for World In Brief, derived from the WBS Dictionary (Section 5)

7. Scheduling Notes & Assumptions

- The 12-week schedule is divided into three 4-week months: Month 1 (Weeks 1–4) covers planning, requirements, design, and database work; Month 2 (Weeks 5–8) covers the bulk of backend, AI Agent, and frontend development; Month 3 (Weeks 9–12) covers feature completion, integration, testing, and deployment.
- Phase 4 (Core Backend), Phase 5 (AI Agent), and Phase 6 (Frontend) are scheduled to run partly in parallel from Week 5, which is typical once the database schema (Phase 3) and architecture (Phase 2) are stable — this keeps the AI Agent's Planner/Executor pipeline (FR-18, FR-19) and the frontend's AI Chat panel in step with each other.
- Phase 8 (Integration & Testing) intentionally starts before every Phase 7 feature module finishes, so that integration defects surface while developers are still available, rather than being discovered only in the final week.
- The schedule reserves the final week (Week 12) exclusively for deployment, documentation, demonstration, and project closure, consistent with NFR-16 (zero-downtime deployment) and NFR-24 (containerized, Node.js-compatible hosting).
- This schedule assumes the constraints in SRS Section 2.5 hold throughout — in particular, that a machine capable of running Llama 3.2 through Ollama is available from the start of Phase 5 (SRS Section 2.6).